Buổi 09 (3 tiết) – Chương 4: Thiết kế giao diện (StyleSheet/Flexbox) + Lab (4.1–4.9)

1 Chương 4: Thiết kế giao diện cho ứng dụng mobile

1.1 4.1 Các cách tạo định dạng StyleSheet

Trong React Native, StyleSheet là một abstraction tương tự như CSS Stylesheets. Nó cung cấp một cách để tạo và quản lý các style cho ứng dụng của bạn một cách hiệu quả và có tổ chức. StyleSheet.create giúp tối ưu hóa hiệu suất bằng cách tạo ra các ID style bất biến, tránh việc tạo ra các style object mới mỗi khi component render lại.

Có hai cách chính để tạo định dạng StyleSheet trong React Native:

1. **Sử dụng StyleSheet.create():** Đây là cách được khuyến nghị và phổ biến nhất. Bạn định nghĩa một object JavaScript, trong đó các key là tên style và các value là các style object (tương tự như CSS trong JavaScript). Sau đó, bạn truyền object này vào hàm StyleSheet.create() để tạo ra một stylesheet object.

   Ví dụ:

   ``javascript import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({ container: { flex: 1, backgroundColor: '#f0f0f0', alignItems: 'center',justifyContent: 'center', }, textTitle: {
fontSize: 24, fontWeight: 'bold', color: 'blue', marginBottom: 10, }, button:{ backgroundColor: '#4CAF50', paddingVertical: 15, paddingHorizontal: 30, bor derRadius: 5, }, buttonText: { color: 'white', fontSize: 16, }, });

export default styles; ``

2. **Inline Styles (Style nội tuyến):** Bạn có thể định nghĩa style trực tiếp như một prop của component. Giá trị của prop style là một object JavaScript chứa các style. Cách này thường được sử dụng cho các style rất cụ thể và ít khi được tái sử dụng.

Ví dụ:

``javascript import React from 'react'; import { View, Text, TouchableOpacity } from 'react-native';

const MyComponent = () => { return (<View style={{ flex: 1, backgroundColor: '#fff', alignItems: 'center',justifyContent: 'center' }}> <Text style={{ fontSize: 20, fontWeight: 'bold', color: 'green', marginBottom: 15 }}>Chào mừng!</Text> <TouchableOpacity style={{ backgroundColor: '#007bff', padding: 10,borderRadius: 3 }}> <Text style={{ color: 'white', fontSize: 14 }}>Nhấn vào đây</Text> </TouchableOpacity> </View>); };

export default MyComponent; ``

Lưu ý: Mặc dù inline styles tiện lợi cho các trường hợp đơn giản, việc sử dụng StyleSheet.create() thường được ưu tiên hơn vì:

- **Hiệu suất:** StyleSheet.create() tạo ra các style ID tĩnh, giúp React tối ưu hóa việc render và tránh tạo các style object mới không cần thiết.

- **Khả năng bảo trì:** Style được tách biệt khỏi logic component, giúp code dễ đọc và dễ quản lý hơn.
- **Tính tái sử dụng:** Các style được định nghĩa trong StyleSheet có thể được sử dụng lại ở nhiều component khác nhau.

1.2 4.2 Sử dụng StyleSheet

Sau khi tạo StyleSheet bằng StyleSheet.create(), bạn có thể sử dụng các style đã định nghĩa cho các component trong phần render() của mình thông qua prop style. Prop style có thể nhận một style object duy nhất hoặc một mảng các style objects (style sau sẽ ghi đè các thuộc tính trùng lặp của style trước).

Cách sử dụng:

1. **Import StyleSheet đã tạo:** Import stylesheet object vào component mà bạn muốn sử dụng.

   ``javascript import React from 'react'; import { View, Text, TouchableOpacity } from 'react-native'; import styles from './Styles'; // Giả sử file Styles.js chứa StyleSheet đã tạo ``

2. **Truyền style cho component:** Sử dụng thuộc tính style và gán cho nó một hoặc một mảng các style từ stylesheet object đã import.

   ``javascript const MyComponent = () => { return (<View style={styles.container}> <Text style={styles.textTitle}>Chào mừng!</Text> <TouchableOpacity style={styles.button}> <Text style={styles.buttonText}>Nhấn vào đây</Text> </TouchableOpacity> </View>); };

   export default MyComponent; ``

3. **Kết hợp nhiều styles:** Bạn có thể truyền một mảng các style objects cho prop style. Điều này hữu ích khi bạn muốn áp dụng các style chung và sau đó ghi đè một vài thuộc tính cụ thể.

   ``javascript <Text style={[styles.textTitle, { color: 'green' }]}>Tiêu đề đã được sửa đổi</Text> ``

   Trong ví dụ này, styles.textTitle sẽ được áp dụng trước, sau đó thuộc tính color sẽ được ghi đè thành 'green'.

1.3 4.3 Export StyleSheet

Để tái sử dụng các style đã định nghĩa trong StyleSheet ở nhiều component khác nhau, bạn cần phải **export** stylesheet object từ file mà nó được tạo. Tương tự như việc export component, bạn có thể sử dụng export default hoặc export(named export).

1. **Default Export:** Đây là cách phổ biến nhất khi một file StyleSheet chứa các style liên quan đến một module hoặc một nhóm component.

   Ví dụ (Styles.js):

   ``javascript import { StyleSheet } from 'react-native';

   const styles = StyleSheet.create({ container: { flex: 1, backgroundColor: '#fff', padding: 20, }, heading: { fontSize: 20, fontWeight: 'bold', marginBottom: 15, }, item: { paddingVertical: 10, borderBottomWidth: 1,borderBottomColor: '#eee', }, });

   export default styles; ``

   Sau đó, bạn có thể import stylesheet này vào bất kỳ component nào bạn muốn sử dụng:

``javascript import React from 'react'; import { View, Text } from 'react-native'; import commonStyles from './Styles';

const MyScreen = () => { return (<View style={commonStyles.container}> <Text style={commonStyles.heading}>Danh sách sản phẩm</Text> <View style={commonStyles.item}> <Text>Sản phẩm 1</Text> </View> <View style={commonStyles.item}> <Text>Sản phẩm 2</Text> </View> </View>); };

export default MyScreen; ``

2. **Named Export:** Bạn có thể export nhiều stylesheet objects từ một file bằng cách sử dụng named export.

   Ví dụ (Theme.js):

   ``javascript import { StyleSheet } from 'react-native';

   export const lightTheme = StyleSheet.create({ background: '#f0f0f0', text: '#333', primaryButton: { backgroundColor: '#007bff', color: 'white', }, });

   export const darkTheme = StyleSheet.create({ background: '#333', text: '#f0f0f0', primaryButton: { backgroundColor: '#00aaff', color: '#222', }, }); ``

   Sau đó, bạn có thể import các theme cụ thể vào component:

   ``javascript import React from 'react'; import { View, Text, TouchableOpacity } from 'react-native'; import { lightTheme } from './Theme';

   const MyComponent = () => { return (<View style={{ flex: 1, backgroundColor: lightTheme.background,padding: 20 }}> <Text style={{ color: lightTheme.text, fontSize: 18 }}>Nội dung với theme

sáng</Text> <TouchableOpacity style={lightTheme.primaryButton}> <Text style={{ color:lightTheme.primaryButton.color }}>Nút chính</Text> </TouchableOpacity> </View>); };

export default MyComponent; ``

1.4 4.4 Truyền Styles dạng Props

Trong một số trường hợp, bạn muốn component con có thể tùy chỉnh style dựa trên props được truyền từ component cha. Điều này giúp tạo ra các component linh hoạt và có khả năng tái sử dụng cao hơn. Bạn có thể truyền các style object trực tiếp như một prop xuống component con.

Ví dụ:

Giả sử bạn có một component CustomButton và bạn muốn cho phép người dùng truyền vào style tùy chỉnh cho nút này.

CustomButton Component:

``javascript import React from 'react'; import { TouchableOpacity, Text, StyleSheet } from 'react-native';

const CustomButton = ({ title, onPress, style, textStyle }) => { return (<TouchableOpacity style={[styles.button, style]} onPress={onPress}> <Text style={[styles.buttonText, textStyle]}>{title}</Text> </TouchableOpacity>); };

const styles = StyleSheet.create({ button: { backgroundColor: '#007bff', paddingVertical: 10, paddingHorizontal: 20, borderRadius: 5, }, buttonText: { color: 'white', fontSize: 16, }, });

export default CustomButton; ``

Cách sử dụng CustomButton và truyền style props:

``javascript import React from 'react'; import { View } from 'react-native'; import CustomButton from './CustomButton';

const MyScreen = () => { const customButtonStyle = { backgroundColor: '#4CAF50', borderColor: 'green', borderWidth: 1, };

const customTextStyle = { fontWeight: 'bold', fontSize: 18, };

return (<View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}> <CustomButton title="Nút mặc định" onPress={() => alert('Mặc định')} /> <CustomButton title="Nút tùy chỉnh" onPress={() => alert('Tùy chỉnh')} style={customButtonStyle} textStyle={customTextStyle} /> </View>); };

export default MyScreen; ``

Trong ví dụ này:

- Component CustomButton nhận hai props style và textStyle.
- Trong phần render, các style mặc định từ styles được kết hợp với các style được truyền qua props bằng cách sử dụng một mảng trong prop style của TouchableOpacity và Text. Các style được truyền sau sẽ ghi đè các style trùng lặp từ styles.
- Trong MyScreen, khi sử dụng CustomButton, chúng ta truyền các style objects customButtonStyle và customTextStyle để tùy chỉnh giao diện của nút thứ hai.

1.5 4.5 Chia sẻ Styles

Việc chia sẻ styles giúp duy trì tính nhất quán trong giao diện ứng dụng và giảm thiểu việc lặp lại code. Có nhiều cách để chia sẻ styles trong React Native:

1. **Export và Import StyleSheet:** Đây là cách cơ bản nhất. Bạn tạo một file JavaScript chứa StyleSheet.create() và sau đó import stylesheet object này vào các component cần sử dụng. (Đã trình bày ở mục 4.3).

2. **Tạo một file theme.js hoặc constants.js:** Bạn có thể tạo một file riêng biệt để định nghĩa các style chung, màu sắc, kích thước font, và các hằng số liên quan đến giao diện. Sau đó, bạn import các giá trị này vào các stylesheet khác hoặc trực tiếp vào component.

   Ví dụ (theme.js):

   ``javascript export const primaryColor = '#007bff'; export const secondaryColor = '#6c757d'; export const textColor = '#333'; export const fontSizeMedium = 16; export const borderRadiusSmall = 3; ``

   Ví dụ (Styles.js):

   ``javascript import { StyleSheet } from 'react-native'; import { primaryColor, textColor, fontSizeMedium, borderRadiusSmall } from './theme';

   const styles = StyleSheet.create({ buttonPrimary: { backgroundColor: primaryColor, paddingVertical: 10, paddingHorizontal: 20, borderRadius: borderRadiusSmall, }, buttonPrimaryText: { color: 'white', fontSize: fontSizeMedium, fontWeight: 'bold', }, textDefault: { color: textColor, fontSize: fontSizeMedium, }, });

   export default styles; ``

3. **Sử dụng Context API hoặc thư viện quản lý state:** Đối với các style phức tạp hơn hoặc các theme có thể thay đổi trong quá trình chạy ứng dụng, bạn có thể sử dụng React Context API hoặc các thư viện quản lý state như Redux hoặc Zustand để chia sẻ thông tin theme và style trên toàn bộ ứng dụng.

4. **Sử dụng thư viện UI Kit:** Có nhiều thư viện UI Kit được xây dựng sẵn cho React Native (ví dụ: NativeBase, React Native Elements). Các thư viện này cung

cấp một tập hợp các component được styled sẵn và có thể tùy chỉnh theme, giúp bạn xây dựng giao diện nhanh chóng và nhất quán.

1.6 4.6 Positioning và Design

React Native sử dụng hệ thống layout dựa trên Flexbox theo mặc định. Flexbox là một mô hình layout mạnh mẽ giúp bạn dễ dàng sắp xếp và căn chỉnh các phần tử trên màn hình, đặc biệt là trong các ứng dụng có giao diện phức tạp và đa dạng kích thước màn hình.

Các thuộc tính Flexbox quan trọng trong React Native:

- **flexDirection:** Xác định hướng chính của các phần tử con trong một container flex. Các giá trị phổ biến là row(theo chiều ngang), column (theo chiều dọc), row-reverse, column-reverse.
- **justifyContent:** Xác định cách các phần tử con được căn chỉnh dọc theo trục chính của container. Các giá trị phổ biến bao gồm flex-start, flex-end, center, space-between, space-around, space-evenly.
- **alignItems:** Xác định cách các phần tử con được căn chỉnh dọc theo trục phụ (trục vuông góc với trục chính). Các giá trị phổ biến bao gồm flex-start, flex-end, center, stretch, baseline.
- **alignSelf:** Cho phép một phần tử con cụ thể ghi đè giá trị alignItems của container cha.
- **flex:** Xác định tỷ lệ không gian mà một phần tử con sẽ chiếm trong container. Một phần tử với flex: 1 sẽ chiếm toàn bộ không gian còn lại.
- **flexWrap:** Xác định xem các phần tử con có nên xuống dòng khi không đủ không gian trên một dòng hay không (wrap, nowrap).

Positioning (Định vị):

Ngoài Flexbox, React Native cũng hỗ trợ các thuộc tính định vị tương tự như CSS:

- **position:** Xác định cách một phần tử được định vị trong tài liệu. Các giá trị phổ biến là relative (vị trí tương đối so với vị trí mặc định của nó) và absolute (vị trí tuyệt đối so với phần tử cha gần nhất có position: 'relative' hoặc so với gốc nếu không có).
- **top, bottom, left, right:** Xác định khoảng cách từ các cạnh của phần tử cha (khi position là absolute hoặc fixed) hoặc từ vị trí mặc định của phần tử (khi position là relative).

Design (Thiết kế):

Ngoài các thuộc tính layout và positioning, React Native cung cấp nhiều thuộc tính style khác để thiết kế giao diện, bao gồm:

- **backgroundColor:** Màu nền.
- **color:** Màu chữ.
- **fontSize, fontWeight, fontStyle:** Thuộc tính font chữ.
- **padding, margin:** Khoảng cách bên trong và bên ngoài phần tử.
- **borderWidth, borderColor, borderRadius:** Đường viền.
- **opacity:** Độ mờ.
- **shadowColor, shadowOffset, shadowOpacity, shadowRadius:** Hiệu ứng bóng đổ (chỉ có trên iOS). Trên Android, bạn có thể sử dụng elevation.
- **transform:** Cho phép xoay, thu phóng, dịch chuyển phần tử.

Việc nắm vững Flexbox và các thuộc tính style khác là rất quan trọng để tạo ra giao diện người dùng đẹp và đáp ứng trên nhiều kích thước màn hình khác nhau trong React Native. Hãy thử nghiệm với các thuộc tính này để hiểu rõ cách chúng hoạt động và xây dựng layout theo ý muốn.

4.7 Câu hỏi ôn tập

6. Nêu 3 lỗi UI thường gặp khi thiết kế layout đa thiết bị và cách khắc phục.

5. Khi nào nên tách style ra file riêng?

4. Nêu cách truyền style qua props và cách merge nhiều style.

3. Position: relative/absolute khác nhau thế nào?

2. Giải thích Flexbox trong RN: flexDirection, justifyContent, alignItems.

1. StyleSheet trong React Native dùng để làm gì? Lợi ích so với inline style?

4.8 Bài tập thực hành

4. Tạo theme màu (primary, text, background) và dùng xuyên suốt.

3. Áp dụng responsive: sử dụng Dimensions hoặc % để UI không vỡ trên màn hình nhỏ.

2. Tạo danh sách (FlatList) hiển thị item dạng card, có ảnh (placeholder) + tiêu đề.

1. Thiết kế màn hình Login theo 2 layout: (1) cột dọc, (2) có logo + form + nút.

4.9 Phiếu bài tập thực hành (Mẫu 04)

Sinh viên thực hiện theo yêu cầu, chụp màn hình/ghi video demo và nộp mã nguồn theo hướng dẫn của giảng viên.

Tên bài thực hành	Chương 4 – Lab 04: Thiết kế UI hoàn chỉnh (Login + List)
Mục tiêu	Áp dụng StyleSheet/Flexbox để thiết kế UI đẹp, responsive và tái sử dụng theme.
Thời lượng gợi ý	150 phút

Chuẩn bị	Đã có project chạy được. Chuẩn bị 1–2 ảnh placeholder (assets/).
Nhiệm vụ thực hành	Tạo màn hình Login: logo, 2 input, nút Login, link 'Forgot password'. Tạo màn hình List: hiển thị 10 item bằng FlatList, mỗi item có ảnh + title + subtitle. Áp dụng theme màu và typography (fontSize, fontWeight). Đảm bảo UI không vỡ khi xoay ngang (landscape) hoặc màn hình nhỏ.
Sản phẩm nộp	Repo + 3 ảnh chụp màn hình (Login, List, landscape hoặc màn hình nhỏ). File theme.js hoặc styles/theme.js.
Tiêu chí đánh giá	Layout đúng yêu cầu (35%). Responsive ổn, dùng Flexbox chuẩn (30%). Theme tái sử dụng (20%). Code gọn, đặt tên rõ (15%).